

flashjet

GPU-native jet clustering with the full merge history
designed for reclustering inside training loops, on padded tensors that stay on the GPU

Chirayu Gupta^{1,*} Sitian Qian² Alexandre De Moor¹

¹Vrije Universiteit Brussel (BE)

²Peking University (CN)

ML4Jets 2026 · Vienna · September 2026

* speaker

- 1 Jet clustering, and flashjet
- 2 Correctness
- 3 Speed
- 4 Downstream: the clustering tree as tagger input

Jet clustering, and flashjet

Reconstruction is moving to accelerators — clustering has not

1. Where the field is

- CMS has run GPUs in the High Level Trigger since Run 3
- **~30 %** of HLT reconstruction is offloaded → **~25 %** less HLT time
- ported: pixel local reco · pixel tracks and vertices · calorimeter local reco

2. What is missing

Jet clustering is **still on the CPU**.

3. Why it stayed there

- sequential recombination is **IRC safe** — that is why it won
- but each merge **depends on the previous one**
- a serial data dependence is the one thing a GPU does badly

4. Why that costs us now

- **reclustering inside a training loop**, so the merge history feeds **substructure** to a tagger
- scanning the **jet radius R**
- **substructure** computed on demand

The tensors are already on the GPU — each of these sends them back to the host and returns, once per call.

What it does

- generalised- k_t clustering — anti- k_t , k_t , Cambridge–Aachen — **transcribed from FastJet and validated against it**
 - runs on the GPU, **many jets or events per launch**
 - returns jets, the **full merge history**, and substructure **as tensors**
- ⇒ designed for **reclustering inside training loops**, on padded $(B, N, 4)$ tensors that **stay on the GPU**

Why it can be done at all

- Cacciari–Salam nearest-neighbour lemma: the smallest d_{ij} in the *whole event* is always some particle's **geometric** nearest neighbour
 - so each particle tracks only **its own** NN, and one merge invalidates $O(1)$ of them
- ⇒ $O(N^2)$ **instead of** $O(N^3)$ — the serial chain stays, but each step becomes cheap and parallel

In the ML ecosystem

- a **PyTorch** call — tensors in, tensors out
- no host $\leftrightarrow$ device copy in a training loop

Two regimes: *event-based* ($N \sim$ hundreds) and *jet-based* ($N \sim$ tens, B large). **This talk is the second.**

Written in **Triton**: autotunes for whatever GPU it finds — no separate CUDA build.

One exponent, three algorithms

The generalised- k_t distance measure:

$$d_{ij} = \min\left(p_{Ti}^{2p}, p_{Tj}^{2p}\right) \frac{\Delta R_{ij}^2}{R^2}, \quad \Delta R_{ij}^2 = (y_i - y_j)^2 + (\phi_i - \phi_j)^2$$
$$d_{iB} = p_{Ti}^{2p}$$

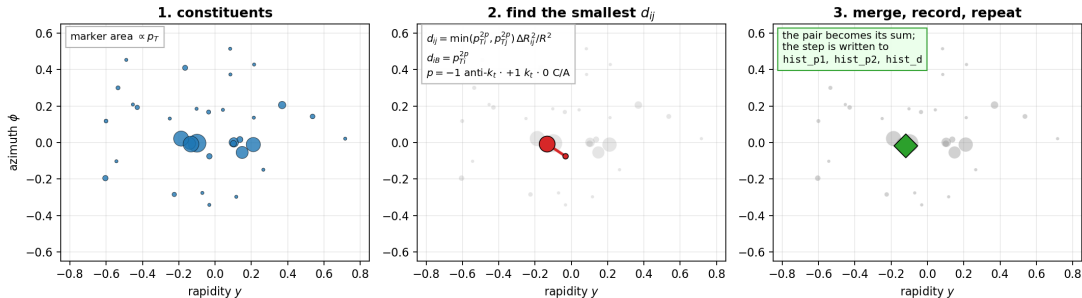
Merge the pair with the smallest d_{ij} ; if the smallest distance is a d_{iB} , call i a jet and remove it.

Only the exponent p changes:

p	algorithm	merges first	tree ordered by	used for
-1	anti- k_t	the hardest pairs	—	finding jets
0	Cambridge–Aachen	the closest pair	angle	grooming, Lund plane
+1	k_t	the softest pair	relative p_T	exclusive subjets

anti- k_t answers *which particles form a jet*. The other two answer *how that jet came apart* — and the **ordering is what the substructure reads**.

Sequential recombination — and what flashjet keeps from it



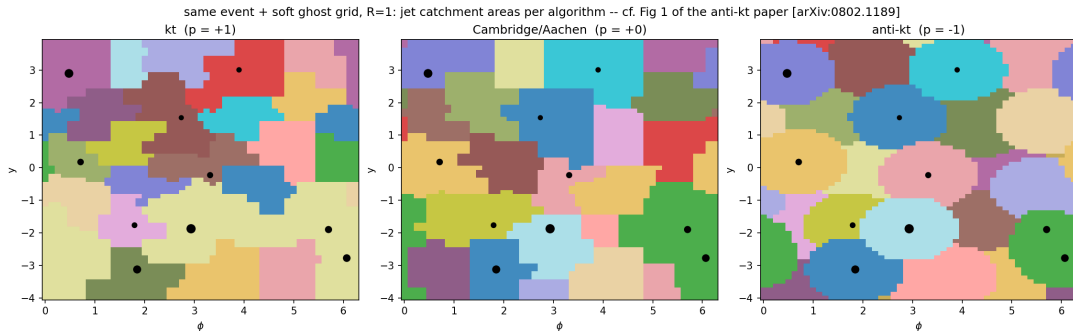
Two distances, computed for every pair:

- d_{ij} — *pair* distance, particle i to particle j
- d_{iB} — *beam* distance, particle i to the beam

Then repeat until nothing is left:

- 1 find the smallest of all distances
- 2 if it is a d_{ij} : **merge** i and j
- 3 if it is a d_{iB} : call i a **jet**, remove it

Both distances were defined on the previous slide; the exponent p there selects which algorithm this loop runs.



- jet catchment areas, all three algorithms
- clustered by **flashjet** itself

- k_t , C/A: **ragged**, fluctuating boundaries
 - anti- k_t : **circles of area πR^2** on each hard particle
- $\Rightarrow$ the algorithm's defining result, **reproduced without FastJet**

It returns the tree, not just the jets

Clustering a batch returns **two** things — the jets, and the record of how they were built.

field	shape	what it holds
jet_idx	(B, N)	which jet each particle ended up in
n_jets	$(B,)$	how many jets were found
hist_p1, hist_p2	(B, N)	which pair merged , at each step
hist_child	(B, N)	what they merged <i>into</i>
hist_d	(B, N)	the distance d_{ij} at that step

The tree is the product, not a by-product

The bottom three rows are the **complete binary merge tree**. Groomed mass, z_g , R_g , Lund coordinates and exclusive subjets are not separate algorithms — they are **coordinates on that one tree**. Compute it once, read them all off, **no second clustering pass**.

Three things are read off that tree — F1, F2, F3 — and then we look at it on a real jet.

What is implemented

Clustering

- generalised- k_t : anti- k_t , k_t , C/A
- merge history recorded in-kernel
- many jets clustered in one batched call
- scales from single jets to whole events

Substructure — reads of the history

- **F1** exclusive k_t subjets
- **F2** soft drop / mMDT / mass-drop
- **F3** primary Lund coordinates

All three are pure-torch post-reads: no kernel changes, CPU/GPU identical, negligible cost.

What it does. Undo the last merges of the sequence to expose exactly n subjects — or stop at a d_{cut} .

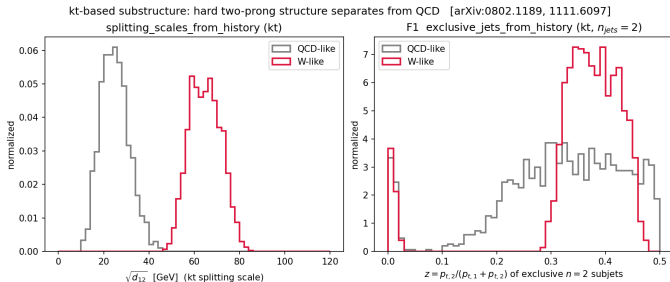
How. The history is already a binary tree. Walk back up it $n - 1$ steps; no reclustering.

Why k_t . k_t merges the softest pair first, so the *last* merges are the hard prongs — exactly what a 2-prong decay leaves behind.

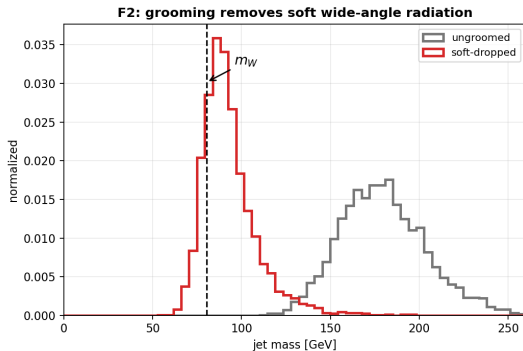
The splitting scales come straight from `hist_d`:

$$\sqrt{d_{12}}, \sqrt{d_{23}}, \dots$$

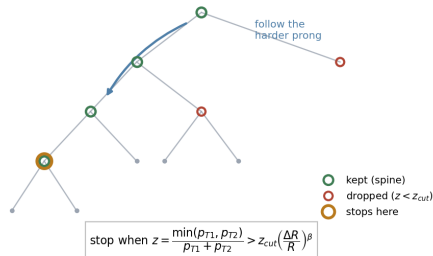
$$\text{with } d_{ij} = \min(p_{Ti}^2, p_{Tj}^2) \frac{\Delta R_{ij}^2}{R^2}$$



Toy W-like (2-prong) vs QCD-like fat jets. **Left:** the k_t splitting scale $\sqrt{d_{12}}$ separates cleanly. **Right:** the momentum balance z of the two exclusive subjects — the W sits near $z \sim 0.4$, QCD spreads to small z .



the soft-drop walk on the C/A tree



Stop when
$$\frac{\min(p_{T1}, p_{T2})}{p_{T1} + p_{T2}} > z_{\text{cut}} \left(\frac{\Delta R_{12}}{R} \right)^\beta$$

Walk down the C/A tree along the harder branch, discarding the softer prong until that condition is met. $\beta = 0$ recovers mMDT.

The walk is $O(\log_2 n)$ on the stored tree — it never touches the constituents again, so grooming is essentially free once clustered.

C/A merge trees of real jets — root at top, constituents at the bottom

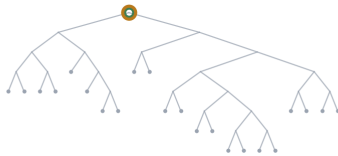
ATLAS Top Tagging Open Data · vertical = declustering depth · horizontal = angular ordering

Light quark / gluon jet



a long spine: soft prong after soft prong
is stripped, and the mass collapses
 $n_{\text{drop}} = 8$ $m: 80 \rightarrow 1 \text{ GeV}$

Boosted top — a clean two-prong core



the very first split is already balanced,
so nothing is groomed away
 $n_{\text{drop}} = 0$ $m: 80 \rightarrow 80 \text{ GeV}$

Boosted top — the full $t \rightarrow bW$ system



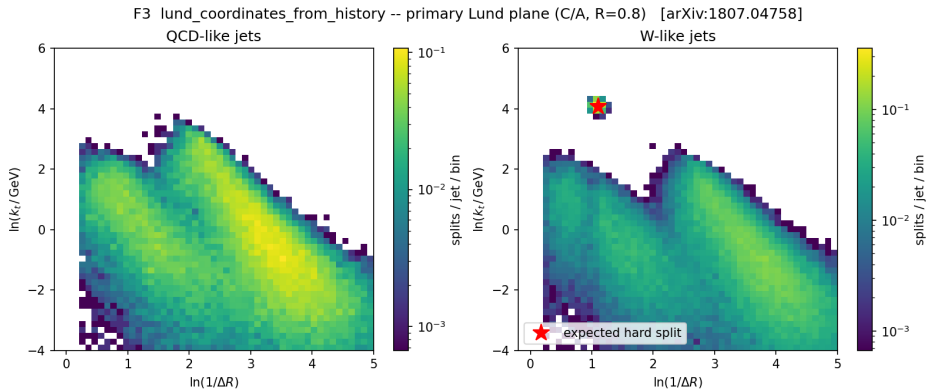
also stops at once, but on a wider split,
so the whole decay is kept
 $n_{\text{drop}} = 0$ $m: 170 \rightarrow 170 \text{ GeV}$

● soft-drop spine ● dropped prong ● grooming stops here ● constituent

A jet without a decay has nothing to stop on. The light-quark/gluon jet is a long *staircase*: soft prong after soft prong is stripped, and its mass collapses to nothing.

ATLAS Open Data, C/A with soft drop ($z_{\text{cut}}=0.1$, $\beta=0$). **The number of drops is itself a discriminant** — and it is read straight off the stored tree, at no extra cost.

A real decay stops at once. Both top jets halt on their very first split, because it is already hard and balanced — so the mass survives.



What. Every primary splitting emits $z = \frac{p_{T2}}{p_{T1} + p_{T2}}$, ΔR , $k_t = p_{T2} \Delta R$, and the plane coordinates $(\ln \frac{1}{\Delta R}, \ln k_t)$.

QCD fills the soft-collinear region smoothly; the W-like sample adds a **localised hard splitting** exactly where the decay is predicted to sit (star).

How. One pass down the primary branch — each node is one point in the plane.

Check. The d -channel ties *exactly* to the splitting scales computed independently.

Correctness

Two kinds of validation

First: toys, where truth is known

On generated inputs the *answer* is known analytically — so any observable landing off the predicted curve is a real bug, with nothing to hide behind.

Input A — toy fat jets — QCD-like and W-like fat jets

Input B — toy parton shower — leading-log parton shower

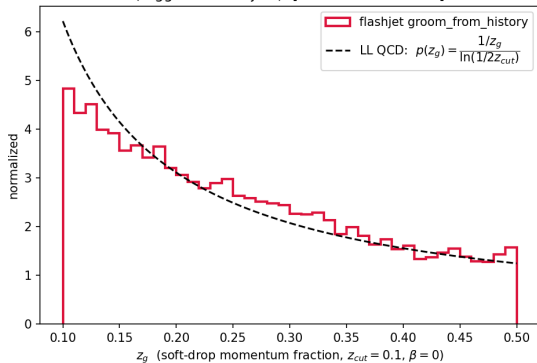
Then: real data, against FastJet

On detector-level jets there is no analytic answer — so the reference is **FastJet on the same events**, and the experiment's own reconstructed jets.

Input C — ATLAS Open Data — large- R jets and full events

The order matters: **toys first** pins the algorithm against a known answer; **real data second** shows it survives detector-level input. A failure at either stage means something different.

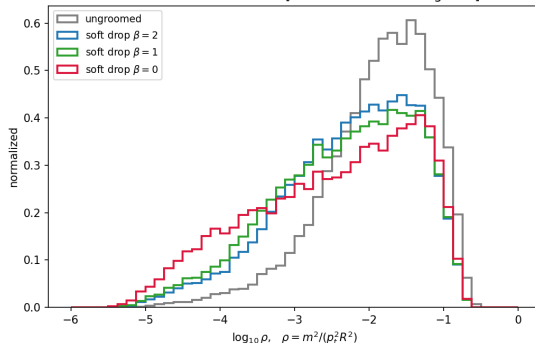
groomed splitting fraction vs the analytic prediction
(tagged 72% of jets) [cf. arXiv:1402.2657]



Soft-drop z_g follows the $1/z$ form predicted at leading-log, recovered from the C/A history.

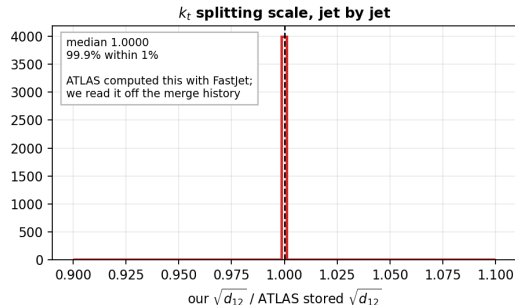
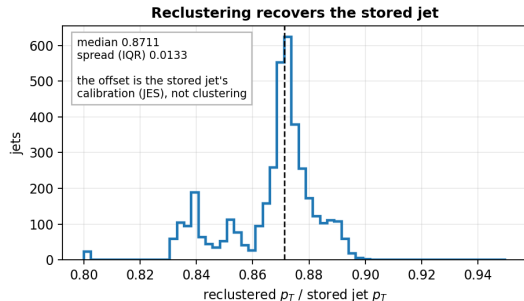
Toy-shower input, so the *prediction* is unambiguous — these test the decoders, not an event generator.

groomed jet mass: stronger grooming (smaller β) pushes the soft-radiation mass down [cf. arXiv:1402.2657 Figs 3-4]



Groomed-mass ordering in β : larger β grooms less, so the mass peak moves up.

Jet-by-jet closure against ATLAS-stored quantities



Left. Reclustering the stored constituents recovers the stored jet with a **0.013 spread**. The 0.87 offset is the stored jet's **calibration**, applied after clustering — it is present before we touch anything.

Right. The k_t splitting scale $\sqrt{d_{12}}$ against the value **ATLAS computed with FastJet** and stored: median **1.0000**, **99.9 %** within 1%. We read ours straight off the merge history.

3 algorithms $\times$ **5 radii** $\times$ **5 multiplicity bins** $\times$ **2 samples**, 20 000 jets per bin, on ATLAS Open Data.

check	result
number of jets found	100.000 % at every one of 150 points
leading-jet p_T within 10^{-4}	1.0000 at 148/150 points
median relative difference	$\sim 7 \times 10^{-8}$ (float32 precision)

The grid is the point

A single working point proves little. Agreement holds across **all three algorithms**, **every radius** from 0.4 to 1.2, and **24 to 123** constituents per jet.

Two independent references

The same events are clustered by **FastJet** for the jets, and every substructure decoder is separately pinned against an **independent NumPy tree-walk**.

A stronger test than agreeing with FastJet.

ATLAS publishes an open dataset carrying both the **calorimeter clusters** and **its own reconstructed jets**. Cluster the ~ 600 clusters per event ourselves, and compare:

n_{jets} vs ATLAS	100.0 % match
mean jets/event	10.96 vs 10.96

This closes the algorithm against **the experiment's own published output** — not against another implementation of the same algorithm.

599.3 clusters/event: the event regime, where the problem is $O(N^2)$.

Speed

Data — ATLAS Open Data, freely presentable

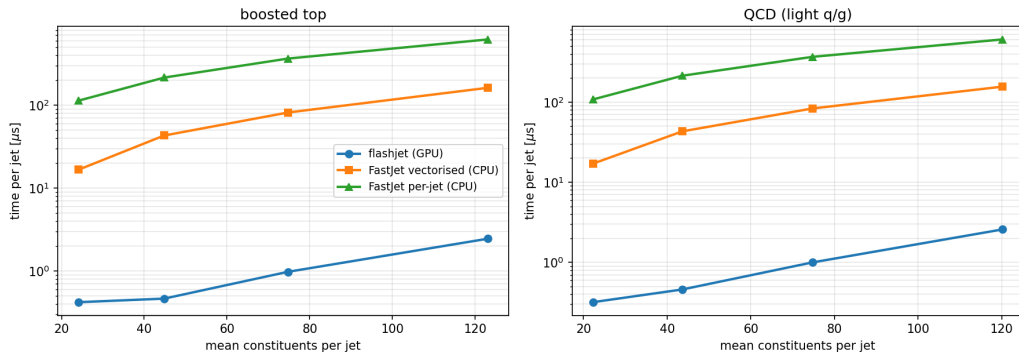
sample	jets	$\langle n_{\text{const}} \rangle$
boosted top	30 000	59.2
QCD (light q/g)	30 000	52.4

Large- R jets with calorimeter-cluster constituents. The grid is **3 algorithms** $\times$ **5 radii** $\times$ **5 multiplicity bins**, 20 000 jets per bin.

Method

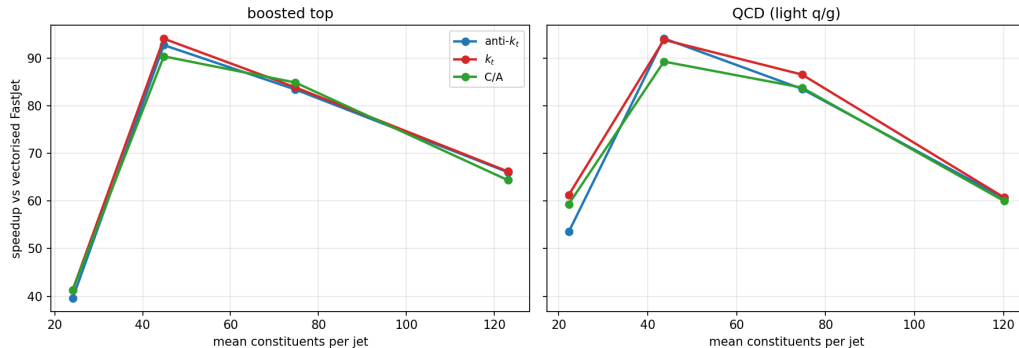
- 1 extract constituents once, save;
- 2 cluster on GPU, time it, save the *same* events;
- 3 cluster those saved events with FastJet;
- 4 **check they agree** — else the timing is meaningless.

Both clusterers read the **same saved file**. Speedups are quoted against the **vectorised** FastJet interface — the faster, fairer baseline.

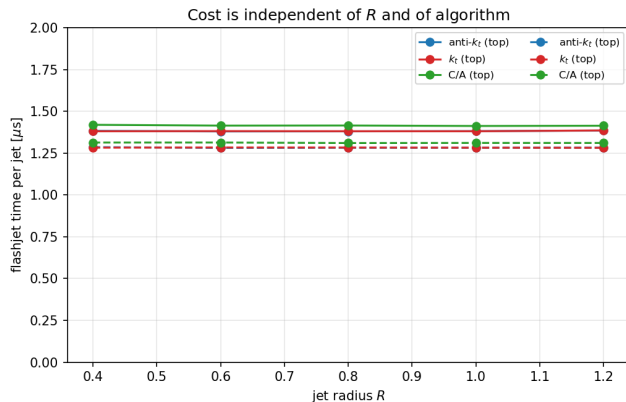
ATLAS Top Tagging Open Data — anti- k_t $R = 1.0$ 

Log scale — each gridline is $10\times$. flashjet stays nearly flat while both FastJet interfaces climb steeply with multiplicity: **0.35–2.7 μs per jet** against **17–155 μs** for vectorised FastJet, and **110–620 μs** per jet one at a time. Benchmarked on an NVIDIA V100-class GPU.

Speedup vs multiplicity — $R = 1.0$ (all points 100% n_{jets} agreement)



All three algorithms, both samples, $R = 1.0$. **39–99 $\times$** over vectorised FastJet across the grid (median **64 $\times$**), and every point has **100 %** n_{jets} agreement. Busier jets $\rightarrow$ bigger win — the regime that matters for boosted-object physics.



Across $R = 0.4 \rightarrow 1.2$: a **0.3 % spread** over a $3\times$ range in R .

R changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti- k_t , k_t and C/A agree to within $\sim 4\%$ at every multiplicity.

k_t and C/A cost the same as anti- k_t .

Using it

```
import flashjet

out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
```

argument	type	
p4	$(B, N, 4)$ tensor	(p_x, p_y, p_z, E) ; CPU or CUDA
mask	(B, N) bool	marks real constituents in a padded batch
R	float	jet radius
algorithm	str	"antikt" "kt" "cambridge"
p	float	generalised- k_t exponent (overrides algorithm)

out.n_jets	# (B,)	jets found
out.jets_p4(p4)	# (B,J,4)	jet four-momenta
out.splitting_scales()	#	$\sqrt{d_{12}}$, $\sqrt{d_{23}}$, ...
out.exclusive_jets(n_jets=3)	# F1	exclusive kt subjects
out.groomed_jets(p4, R, z_cut=0.1, beta=0.0)	# F2	soft drop / mMDT
out.lund_coordinates(p4, R)	# F3	primary Lund plane

The same call runs unchanged on CPU and GPU — the backend is chosen from the problem size, not by the caller.

Downstream: the clustering tree as tagger input

Does the tree help a tagger? — setup

The question. Gouskos & Maier [PLuM, 2605.26821] report that explicit Lund-plane splittings help a transformer tagger. We test that on a **full 10-class** training.

Model. Particle Transformer [2202.03772] — 8 encoder + 2 class-attention blocks, 8 heads, embed 128. **2.144 M** params baseline, **2.194 M** with the feature block (+0.051 M).

Data. JetClass, 10 classes, **~20.05 M** test jets (~ 2.0 M/class), 128 particles/jet. All arms read **identical** shards; features computed live.

Ranger, lr 10^{-3} , batch 512, 10^6 iterations, checkpoint every 20k. Uncertainties: bootstrap of the test set, 16–84th percentile.

The feature arms

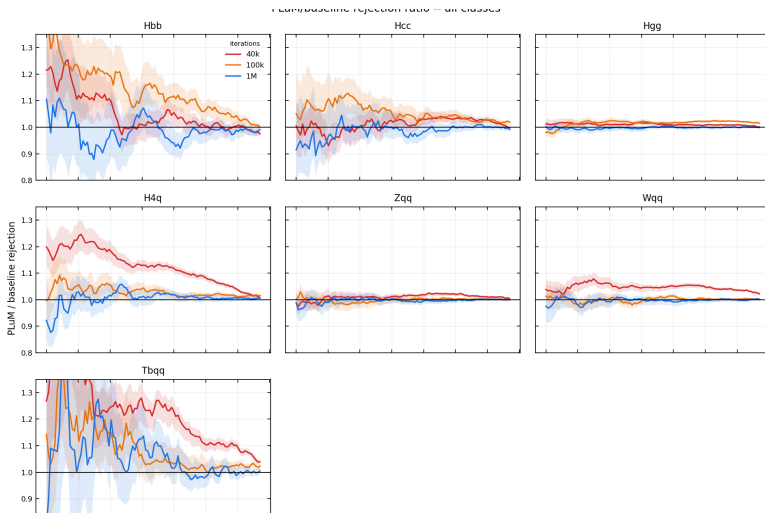
- **PLuM** — 48 k_t -ordered Lund splittings, $[\ln k_t, \ln 1/\Delta R, \ln z]$, embedded and **concatenated** as extra tokens (176 vs 128)
- **subject** — n -body subject mass, p_T fraction, n_{const}
- **CA5** — per-particle C/A branch-point features

Cost (wall clock per step)

PLuM	1.59×
subject	1.42×
CA5	1.22×

One seed per arm — see the caveat on the next slide.

Does the tree help a tagger? — result



PLuM/baseline rejection ratio; unpaired-bootstrap bands.

Test accuracy ~ 20.05 M jets

	40k	100k	1M
baseline	83.431	84.632	86.212
PLuM	83.428	84.697	86.206
Δ	-0.003	+0.065	-0.006

What we see

An early gain in every class, gone by 1 M. Wqq has the *fewest* splittings (21.6/jet) yet gains at $\sim 9\sigma$: **not confined to complex or *b*-jets**. At convergence: **null-or-worse**.

What we do *not* claim

- **No mechanism** — *b*-specificity excluded.
- **One seed per arm** — a second seed's spread exceeded the effect.
- Their Hbb reproduced at 100k, **not** at convergence.

- 1 **Clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjects, soft drop, Lund coordinates — comes free as array reads. Built for **reclustering inside a training loop**, on tensors that never leave the GPU.
- 2 **Gives the same jets as FastJet**. 150/150 grid points at 100 % n_{jets} agreement, across anti- k_t/k_t /C-A, $R = 0.4\text{--}1.2$ and 24–123 constituents per jet.
- 3 **Reproduces the experiment's own reconstructed jets** exactly, at ~ 600 clusters per event.
- 4 **39–99 $\times$ faster** than vectorised FastJet, and the advantage *grows* with jet complexity. Cost is independent of R and of algorithm.
- 5 Still a **prototype**, with meaningful headroom left.

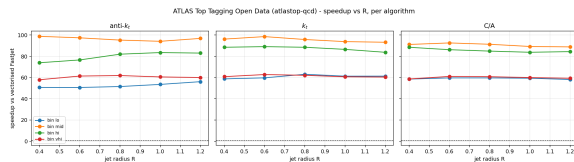
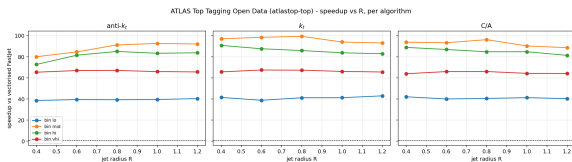
Thank you

Questions?

Benchmarks: ATLAS Open Data (CC0).

Physics closures: analytic predictions on toy showers.

Backup

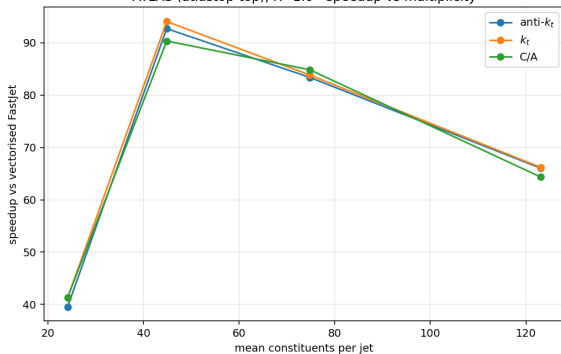


boosted top

Flat in R for all three algorithms; ordering is by multiplicity bin.

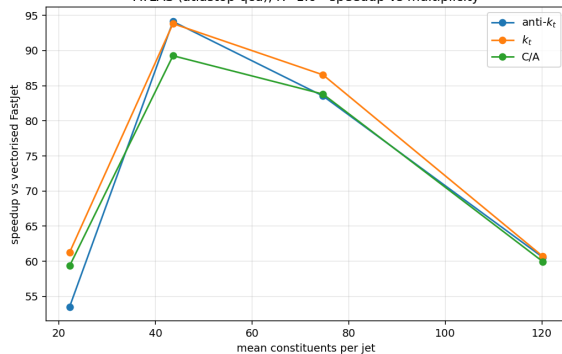
QCD (light q/g)

ATLAS (atlastop-top), R=1.0 - speedup vs multiplicity



boosted top

ATLAS (atlastop-qcd), R=1.0 - speedup vs multiplicity



QCD (light q/g)

Backup — the numbers

sample	alg	$\langle n \rangle$	flashjet [$\mu\text{s}/\text{jet}$]	FastJet vec. [$\mu\text{s}/\text{jet}$]	speedup
top	anti- k_t	24.1	0.427	16.5	39×
top	anti- k_t	44.8	0.464	43.0	93 ×
top	C/A	74.7	1.000	84.8	85×
top	C/A	123.1	2.565	165.1	64×
qcd	anti- k_t	22.2	0.346	17.5	51×
qcd	anti- k_t	43.6	0.463	45.7	99 ×
qcd	anti- k_t	74.7	1.215	89.8	74×
qcd	anti- k_t	120.2	2.673	154.9	58×

39–99× over vectorised FastJet (median 64×), with 100 % jet-count agreement everywhere. Benchmarked on an **NVIDIA V100-class** GPU.

$R = 1.0$ rows shown. Against the per-jet FastJet interface the gap is roughly *two orders of magnitude*.

Backup — where the time goes

Nsight Systems — share of GPU time

kernel	% GPU	avg/call
_cluster_large_kernel	97.6	5.47 ms
_decode_kernel	1.5	82 μ s
torch glue	< 1	—

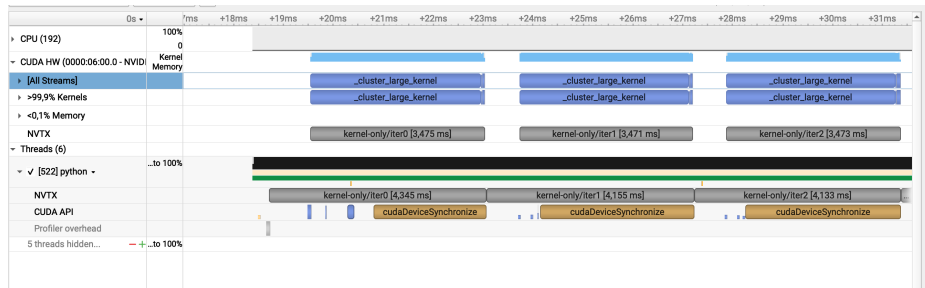
No hidden overhead — essentially all the time is real clustering work, and the memory row of the timeline is empty.

Nsight Compute — hardware counters

metric	value
DRAM throughput	0.02 %
Compute (SM) throughput	15.4 %
Registers / thread	168
Theoretical occupancy	18.75 %
Achieved occupancy	6.25 %
Waves per SM	0.32

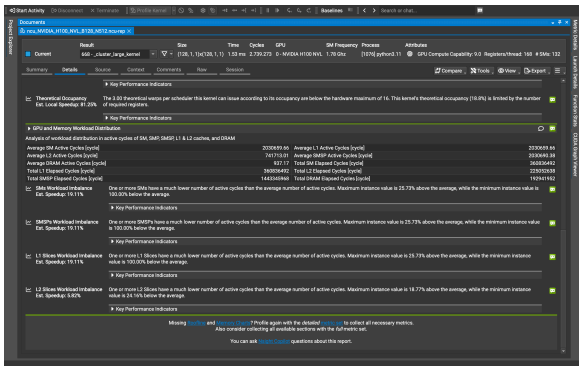
Not memory- or compute-bound — the GPU is simply **not filled**.

Backup — the GPU timeline



Nsight Systems, three consecutive iterations. Each block is one clustering call; the tool labels the rows “>99.9 % Kernels” and “<0.1 % Memory”. The kernels run back to back with nothing in between and the memory row is empty — no host/device shuffling while clustering runs. That is what makes the speedup real rather than an artefact of where the timer was started.

Backup — the profiler names the bottleneck



The tool's own speedup estimates:

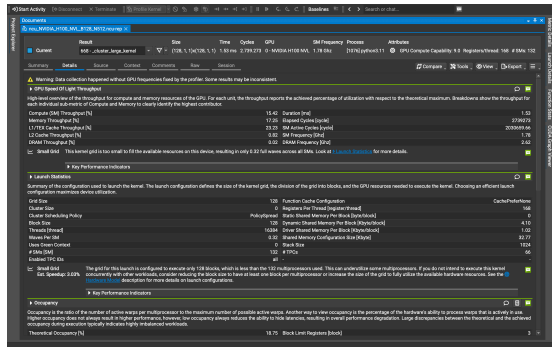
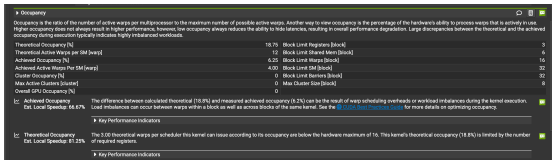
finding	est. speedup
Theoretical occupancy (registers)	81 %
Achieved occupancy	67 %
SM workload imbalance	19 %

"This kernel grid is too small to fill the available resources on this device, resulting in only 0.32 full waves across all SMs."

Grid size **128 blocks** on a **132-SM** device — fewer blocks than the GPU has processors.

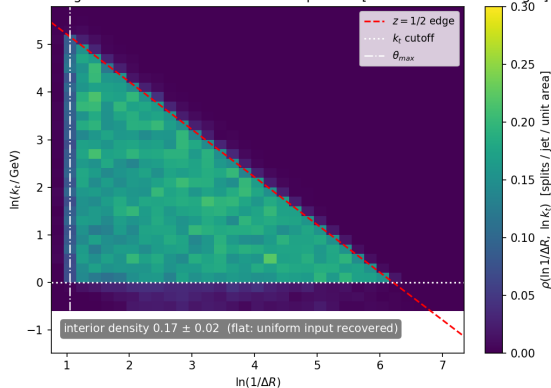
~**1.7–1.8×** more available from launch configuration and register budget — **not** from the algorithm.

Backup — occupancy detail



Register pressure (168/thread) caps theoretical occupancy at 18.75 %; achieved is 6.25 %. With 0.32 waves/SM the kernel does not fill the GPU even once, while DRAM throughput is 0.02 % — this is parallelism starvation, not a bandwidth problem. Both are configuration inherited from smaller GPUs, not properties of the algorithm.

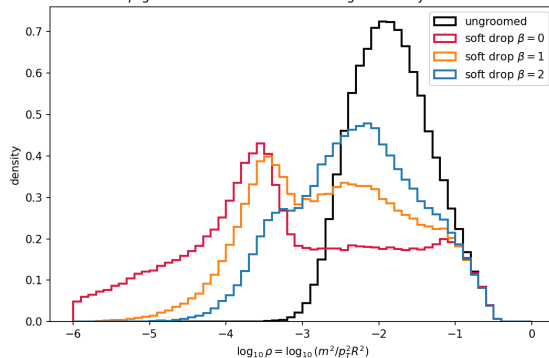
primary Lund plane of the toy shower (C/A, R=0.4):
m-in-triangle emissions come back as a flat plateau [cf. arXiv:1807.04758 Fig 2]



The Lund plane is populated **uniformly** inside the kinematic **triangle** — the leading-log expectation.

Independent NumPy tree-walks pin every decoder, and the d -channel of the Lund output ties *exactly* to the splitting scales computed separately.

Soft-drop β -ordering on REAL QCD jets (164292 jets, $z_{cut} = 0.1$)
smaller β grooms harder — same ordering as the toy-shower closure



The soft-drop β -family orders as predicted: larger β grooms less.

Backup — Input A: the toy fat jets

Purpose. QCD vs W is a *known* truth, so any observable that fails to separate them — or lands off the analytic curve — is a real bug.

Two single-fat-jet samples at $p_T \approx 500$ GeV, $R = 0.8$:

- **QCD-like:** one hard collinear core (92 % of p_T , $\sigma_{y,\phi} = 0.06$) + soft wide-angle radiation (8 %, $\sigma = 0.30$)
- **W-like:** two hard prongs with pair mass $m = \sqrt{z(1-z)} p_T$, $\Delta R = 80.4$ GeV, $z \in [0.30, 0.45]$, random orientation, + 6 % soft contamination

Each “spray” is n massive pions with exponential p_T fractions:

```
frac = rng.exponential(1., n); frac /= frac.sum()
pt    = pt_total * frac
y, phi = rng.normal(y0, sigma, n), rng.normal(phi0, sigma, n)
mt     = np.sqrt(M_PI**2 + pt**2)
```

1500 events per sample. `flashjet` has **no event generator** — it clusters four-vectors; these toys live in the plot scripts, outside the library.

Backup — Input B: the toy parton shower

A **primary, fixed-coupling, leading-log** shower: emissions sampled **uniformly in the Lund triangle** with density $\bar{\alpha}$ per unit $(\ln 1/\theta, \ln k_t)$ area, off a hard spine.

$$\theta = e^{-u}, \quad k_t = e^v, \quad z = \frac{k_t}{p_{T0} \theta}$$

keep $v \leq \ln(p_{T0}/2) - u$ (i.e. $z \leq \frac{1}{2}$), $k_t > k_t^{\min}$

Why this is the right test

The generator's density is *uniform in the triangle by construction*, so the leading-log predictions — the $1/z$ form of z_g , the β -ordering of the groomed mass — are unambiguous. Landing on them tests the **decoders**, not the shower.

Jet areas use the anti- k_t -paper construction: one hard event plus a dense grid of **infinitely soft ghosts** ($p_T = 10^{-8}$); the ghosts each algorithm sweeps up trace its catchment area.

$\bar{\alpha} = 0.25$, $p_{T0} = 1$ TeV, $R = 0.4$, 20 000 showers; fixed seed, fully re-runnable.

Backup — the validation ladder

rung	input	reference	result
unit tests	—	independent NumPy tree-walks	129 pass, incl. GPU paths
anti- k_t jet areas	toy B	the anti- k_t paper's own construction	rigid cones, $\rightarrow \pi R^2$
k_t substructure	toy A	known 2-prong vs QCD truth	clean separation
soft-drop z_g	toy B	leading-log $1/z$ prediction	on the curve
groomed mass vs β	toy B	Soft Drop Figs. 3–4	ordering reproduced
Lund plane	toy A	primary-Lund construction	hard split where predicted
jet-by-jet kinematics	real C	FastJet, same events	150/150 points, 100 % n_{jets}
full-event clustering	real C	the experiment's own jets	100 % match, ~ 600 clusters/ev

Each rung is closed against something independent — an analytic prediction, a separate NumPy implementation, or FastJet itself. No rung is validated against another part of flashjet.

- H. Qu, C. Li, S. Qian, *Particle Transformer for Jet Tagging*, arXiv:**2202.03772**. (also the JetClass dataset; Zenodo 10.5281/zenodo.6619768)
- L. Gouskos, B. Maier, *Particle-Lund Multimodality in Jet Taggers*, arXiv:**2605.26821**.
- H. Qu, L. Gouskos, *Jet tagging via particle clouds* (ParticleNet), Phys. Rev. D **101** (2020) 056019, arXiv:1902.08570.
- F. A. Dreyer, G. P. Salam, G. Soyez, *The Lund Jet Plane*, JHEP **12** (2018) 064, arXiv:1807.04758.
- F. A. Dreyer, H. Qu, *Jet tagging in the Lund plane with graph networks*, JHEP **03** (2021) 052, arXiv:2012.08526.
- A. J. Larkoski, S. Marzani, G. Soyez, J. Thaler, *Soft Drop*, JHEP **05** (2014) 146, arXiv:1402.2657.
- M. Cacciari, G. P. Salam, G. Soyez, *The anti- k_t jet clustering algorithm*, JHEP **04** (2008) 063, arXiv:0802.1189; *FastJet user manual*, Eur. Phys. J. C **72** (2012) 1896, arXiv:1111.6097.
- S. Catani, Yu. L. Dokshitzer, M. H. Seymour, B. R. Webber, Nucl. Phys. B **406** (1993) 187 (k_t); Yu. L. Dokshitzer *et al.*, hep-ph/9707323 and M. Wobisch, T. Wengler, hep-ph/9907280 (Cambridge/Aachen).
- J. de Favereau *et al.*, *DELPHES 3*, JHEP **02** (2014) 057, arXiv:1307.6346.

All benchmark data is **ATLAS Open Data** (CC0), read over the public redirector — no grid certificate required.

dataset	used for
ATLAS Top Tagging Open Data	timing + FastJet agreement
2020 ATLAS Jet Reconstruction	closure vs the experiment's own jets

Constituents are calorimeter clusters, massless. Physics closures use toy showers compared against leading-log analytic predictions, so the prediction being tested is unambiguous.